

Git sees that we modified *hello_git.py* ❶, and we can commit the change if we want to. But this time, instead of committing the change, we'll go back to the last commit when we knew our project was working. We won't do anything to *hello_git.py*: we won't delete the line or use the Undo feature in the text editor. Instead, enter the following commands in your terminal session:

```
git_practice$ git restore .
git_practice$ git status
On branch main
nothing to commit, working tree clean
git_practice$
```

The command `git restore filename` allows you to abandon all changes since the last commit in a specific file. The command `git restore .` abandons all changes made in all files since the last commit; this action restores the project to the last committed state.

When you return to your text editor, you'll see that *hello_git.py* has changed back to this:

```
print("Hello Git world!")
print("Hello everyone.")
```

Although going back to a previous state might seem trivial in this simple project, if we were working on a large project with dozens of modified files, all the files that had changed since the last commit would be restored. This feature is incredibly useful: you can make as many changes as you want when implementing a new feature, and if they don't work, you can discard them without affecting the project. You don't have to remember those changes and manually undo them. Git does all of that for you.

NOTE

You might have to refresh the file in your editor to see the restored version.

Checking Out Previous Commits

You can revisit any commit in your log, using the checkout command, by using the first six characters of a reference ID. After checking out and reviewing an earlier commit, you can return to the latest commit or abandon your recent work and pick up development from the earlier commit:

```
git_practice$ git log --pretty=oneline
945fa13af128a266d0114eebb7a3276f7d58ecd2 (HEAD -> main) Extended greeting.
cea13ddc51b885d05a410201a54faf20e0d2e246 Started project.
git_practice$ git checkout cea13d
Note: switching to 'cea13d'.
```

- ❶ You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.